Software Services- Learn To Create

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

SQL Subquery Practice Workbook

60 Questions · Questions & Expected Outputs

30 Easy · 15 Intermediate · 15 Hard

Section 1 — Sample Tables

All 60 questions use the five tables below. Study the data carefully before attempting each question.

1.1 employees (20 rows)

emp_id	emp_name	department	salary	manager_id	hire_year
101	Alice Johnson	IT	72000	NULL	2018
102	Bob Smith	IT	65000	101	2019
103	Carol White	HR	58000	NULL	2017
104	David Brown	HR	54000	103	2020
105	Eva Green	Finance	80000	NULL	2016
106	Frank Black	Finance	75000	105	2018
107	Grace Lee	IT	68000	101	2021
108	Henry Wilson	Marketing	60000	NULL	2017
109	Ivy Turner	Marketing	56000	108	2022
110	Jack Davis	Finance	70000	105	2019
111	Karen Moore	IT	63000	101	2020
112	Leo Harris	HR	50000	103	2021
113	Mia Clark	Marketing	62000	108	2018
114	Nate Lewis	Finance	78000	105	2019
115	Olivia Walker	IT	71000	101	2017
116	Paul Hall	HR	53000	103	2020
117	Quinn Young	Marketing	59000	108	2021
118	Rachel King	Finance	82000	105	2016
119	Sam Scott	IT	66000	101	2022
120	Tina Adams	HR	55000	103	2019

1.2 orders (20 rows)

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

order_id	customer_id	product_id	amount	order_date	status
1001	201	301	1500	2024-01-05	Delivered
1002	202	302	2300	2024-01-12	Delivered
1003	203	303	800	2024-02-03	Delivered
1004	201	304	3200	2024-02-14	Delivered
1005	204	301	1100	2024-03-01	Pending
1006	205	302	4500	2024-03-15	Delivered
1007	202	303	950	2024-03-22	Cancelled
1008	206	304	2100	2024-04-05	Delivered
1009	203	301	1750	2024-04-18	Pending
1010	207	302	3300	2024-04-30	Delivered
1011	204	303	680	2024-05-10	Cancelled
1012	208	304	2900	2024-05-22	Delivered
1013	205	301	1200	2024-06-08	Delivered
1014	201	302	2700	2024-06-15	Pending
1015	209	303	500	2024-06-28	Delivered
1016	206	304	3800	2024-07-07	Delivered
1017	207	301	1600	2024-07-19	Delivered
1018	210	302	4100	2024-07-25	Delivered
1019	208	303	720	2024-08-02	Cancelled
1020	209	304	2200	2024-08-14	Delivered

1.3 products (20 rows)

product_id	product_name	category	price	stock_qty
301	Laptop	Electronics	55000	30
302	Smartphone	Electronics	22000	80
303	Headphones	Electronics	3500	120
304	Monitor	Electronics	18000	45
305	Keyboard	Accessories	1500	200
306	Mouse	Accessories	900	250
307	Desk Chair	Furniture	8500	40
308	Standing Desk	Furniture	22000	15
309	Webcam	Electronics	4200	70
310	USB Hub	Accessories	1200	180
311	Notebook	Stationery	150	500
312	Pen Set	Stationery	250	600

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

product_id	product_name	category	price	stock_qty
313	Printer	Electronics	12000	25
314	Scanner	Electronics	9000	20
315	External SSD	Electronics	7500	60
316	Lamp	Furniture	2200	90
317	Whiteboard	Office Supplies	3800	35
318	Cable Organiser	Accessories	600	300
319	Power Bank	Electronics	2800	100
320	Smart Watch	Electronics	15000	55

1.4 customers (20 rows)

customer_id	customer_name	city	country	join_year
201	Aarav Mehta	Mumbai	India	2021
202	Priya Sharma	Delhi	India	2020
203	Rohan Gupta	Pune	India	2022
204	Sunita Patel	Ahmedabad	India	2021
205	Vikram Rao	Bangalore	India	2019
206	Neha Singh	Chennai	India	2023
207	Arjun Kumar	Hyderabad	India	2020
208	Deepa Nair	Kochi	India	2022
209	Sanjay Joshi	Jaipur	India	2021
210	Meera Iyer	Coimbatore	India	2023
211	Ravi Verma	Lucknow	India	2020
212	Anita Das	Kolkata	India	2019
213	Kiran Reddy	Vizag	India	2022
214	Pooja Shah	Surat	India	2021
215	Amit Bose	Nagpur	India	2023
216	Tara Menon	Trivandrum	India	2020
217	Nikhil Kulkarni	Nashik	India	2022
218	Swati Chatterjee	Bhopal	India	2019
219	Rahul Pandey	Patna	India	2021
220	Divya Tiwari	Indore	India	2023

1.5 departments (4 rows)

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

dept_id	dept_name	budget	location	head_id
1	IT	500000	Bangalore	101
2	HR	200000	Mumbai	103
3	Finance	600000	Delhi	105
4	Marketing	300000	Chennai	108

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

Section 2 — Easy Questions (Q1–Q30)

📌 Single-Row Subquery · Multi-Row Subquery (IN / NOT IN / ANY / ALL) · EXISTS / NOT EXISTS

Q1. Find the names, departments, and salaries of all employees whose salary is greater than the average salary of all employees in the company. The company-wide average salary should be computed inside a subquery.

Easy | Topic: Single-Row Subquery

Expected Output:

emp_name	department	salary
Rachel King	Finance	82000
Eva Green	Finance	80000
Nate Lewis	Finance	78000
Frank Black	Finance	75000
Alice Johnson	IT	72000
Olivia Walker	IT	71000
Jack Davis	Finance	70000
Grace Lee	IT	68000
Sam Scott	IT	66000
Bob Smith	IT	65000

Q2. Display the emp_id, emp_name, department, and salary of the single highest-paid employee in the entire company. Use a subquery that returns the maximum salary, and filter the main query to match it.

Easy | Topic: Single-Row Subquery

Expected Output:

emp_id	emp_name	department	salary
118	Rachel King	Finance	82000

Q3. Display the emp_id, emp_name, department, and salary of the employee with the lowest salary in the entire company.

Easy | Topic: Single-Row Subquery

Expected Output:

emp_id	emp_name	department	salary
112	Leo Harris	HR	50000

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

Q4. Find all employees (other than Rachel King herself) who were hired in the same year as Rachel King. Return their emp_name, department, and hire_year. Use a single-row subquery to find Rachel King's hire year.

Easy | Topic: Single-Row Subquery

Expected Output:

emp_name	department	hire_year
Eva Green	Finance	2016

Q5. List all products whose price is greater than the average price across all products in the entire products table. Show product_name, category, and price, ordered by price descending.

Easy | Topic: Single-Row Subquery

Expected Output:

product_name	category	price
Laptop	Electronics	55000
Smartphone	Electronics	22000
Standing Desk	Furniture	22000
Monitor	Electronics	18000
Smart Watch	Electronics	15000
Printer	Electronics	12000

Q6. Retrieve the product_id, product_name, category, and price of the single most expensive product in the entire products table.

Easy | Topic: Single-Row Subquery

Expected Output:

product_id	product_name	category	price
301	Laptop	Electronics	55000

Q7. Retrieve the product_id, product_name, category, and price of the cheapest product in the entire products table.

Easy | Topic: Single-Row Subquery

Expected Output:

product_id	product_name	category	price
311	Notebook	Stationery	150

Q8. List all orders whose amount is greater than the average order amount across all orders in the orders table. Show order_id, customer_id, and amount, ordered by amount descending.

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

Easy | Topic: Single-Row Subquery

Expected Output:

order_id	customer_id	amount
1006	205	4500.0
1018	210	4100.0
1016	206	3800.0
1010	207	3300.0
1004	201	3200.0
1012	208	2900.0
1014	201	2700.0
1002	202	2300.0
1020	209	2200.0
1008	206	2100.0

Q9. Find the order_id, customer_id, amount, and order_date of the single most expensive order ever placed (the order with the highest amount in the entire orders table).

Easy | Topic: Single-Row Subquery

Expected Output:

order_id	customer_id	amount	order_date
1006	205	4500.0	2024-03-15

Q10. Using the IN operator with a subquery, find all customers who have placed at least one order. Return customer_id, customer_name, and city, ordered by customer_id.

Easy | Topic: Multi-Row Subquery (IN)

Expected Output:

customer_id	customer_name	city
201	Aarav Mehta	Mumbai
202	Priya Sharma	Delhi
203	Rohan Gupta	Pune
204	Sunita Patel	Ahmedabad
205	Vikram Rao	Bangalore
206	Neha Singh	Chennai
207	Arjun Kumar	Hyderabad
208	Deepa Nair	Kochi
209	Sanjay Joshi	Jaipur

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

customer_id	customer_name	city
210	Meera Iyer	Coimbatore

Q11. Using the NOT IN operator with a subquery, find all customers who have NEVER placed any order. Return customer_id, customer_name, and city, ordered by customer_id.

Easy | Topic: Multi-Row Subquery (NOT IN)

Expected Output:

customer_id	customer_name	city
211	Ravi Verma	Lucknow
212	Anita Das	Kolkata
213	Kiran Reddy	Vizag
214	Pooja Shah	Surat
215	Amit Bose	Nagpur
216	Tara Menon	Trivandrum
217	Nikhil Kulkarni	Nashik
218	Swati Chatterjee	Bhopal
219	Rahul Pandey	Patna
220	Divya Tiwari	Indore

Q12. Using the IN operator with a subquery on the orders table, find all products that appear in at least one order. Show product_id, product_name, category, and price.

Easy | Topic: Multi-Row Subquery (IN)

Expected Output:

product_id	product_name	category	price
301	Laptop	Electronics	55000
302	Smartphone	Electronics	22000
303	Headphones	Electronics	3500
304	Monitor	Electronics	18000

Q13. Using the NOT IN operator, find all products that have NEVER appeared in any order. Return product_id, product_name, category, and price.

Easy | Topic: Multi-Row Subquery (NOT IN)

Expected Output:

product_id	product_name	category	price
305	Keyboard	Accessories	1500

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

product_id	product_name	category	price
306	Mouse	Accessories	900
307	Desk Chair	Furniture	8500
308	Standing Desk	Furniture	22000
309	Webcam	Electronics	4200
310	USB Hub	Accessories	1200
311	Notebook	Stationery	150
312	Pen Set	Stationery	250
313	Printer	Electronics	12000
314	Scanner	Electronics	9000
315	External SSD	Electronics	7500
316	Lamp	Furniture	2200
317	Whiteboard	Office Supplies	3800
318	Cable Organiser	Accessories	600
319	Power Bank	Electronics	2800
320	Smart Watch	Electronics	15000

Q14. Using an IN subquery against the departments table, retrieve the emp_name and salary of all employees who belong to the Finance department. Order by salary descending. (Do not hardcode the department name in the WHERE clause of the outer query — use a subquery to fetch it.)

Easy | Topic: Multi-Row Subquery (IN)

Expected Output:

emp_name	salary
Rachel King	82000
Eva Green	80000
Nate Lewis	78000
Frank Black	75000
Jack Davis	70000

Q15. Find all employees whose salary is less than the minimum salary of any employee in the Finance department. Return emp_name, department, and salary, ordered by salary descending. (Use a single-row subquery returning MIN salary of Finance.)

Easy | Topic: Single-Row Subquery

Expected Output:

emp_name	department	salary
Grace Lee	IT	68000

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

emp_name	department	salary
Sam Scott	IT	66000
Bob Smith	IT	65000
Karen Moore	IT	63000
Mia Clark	Marketing	62000
Henry Wilson	Marketing	60000
Quinn Young	Marketing	59000
Carol White	HR	58000
Ivy Turner	Marketing	56000
Tina Adams	HR	55000
David Brown	HR	54000
Paul Hall	HR	53000
Leo Harris	HR	50000

Q16. Find the department with the highest budget from the departments table. Return dept_name, budget, and location. Use a single-row subquery that returns MAX(budget).

Easy | Topic: Single-Row Subquery

Expected Output:

dept_name	budget	location
Finance	600000	Delhi

Q17. Using a subquery with GROUP BY and HAVING, find all customers who have placed exactly 1 order. Return customer_id, customer_name, and city.

Easy | Topic: Multi-Row Subquery (IN)

Expected Output:

customer_id	customer_name	city
210	Meera Iyer	Coimbatore

Q18. Using a subquery with GROUP BY and HAVING, find all customers who have placed 2 or more orders. Return customer_id, customer_name, and city, ordered by customer_id.

Easy | Topic: Multi-Row Subquery (IN)

Expected Output:

customer_id	customer_name	city
201	Aarav Mehta	Mumbai
202	Priya Sharma	Delhi

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

customer_id	customer_name	city
203	Rohan Gupta	Pune
204	Sunita Patel	Ahmedabad
205	Vikram Rao	Bangalore
206	Neha Singh	Chennai
207	Arjun Kumar	Hyderabad
208	Deepa Nair	Kochi
209	Sanjay Joshi	Jaipur

Q19. Find all products whose price is greater than the maximum price of any product in the 'Accessories' category. Use a scalar subquery returning MAX price for Accessories. Return product_name, category, and price ordered by price descending.

Easy | Topic: Single-Row Subquery

Expected Output:

product_name	category	price
Laptop	Electronics	55000
Smartphone	Electronics	22000
Standing Desk	Furniture	22000
Monitor	Electronics	18000
Smart Watch	Electronics	15000
Printer	Electronics	12000
Scanner	Electronics	9000
Desk Chair	Furniture	8500
External SSD	Electronics	7500
Webcam	Electronics	4200
Whiteboard	Office Supplies	3800
Headphones	Electronics	3500
Power Bank	Electronics	2800
Lamp	Furniture	2200

Q20. Using an IN subquery, find all orders placed by customers who joined in the year 2021. Return order_id, customer_id, amount, and order_date, ordered by order_date.

Easy | Topic: Multi-Row Subquery (IN)

Expected Output:

order_id	customer_id	amount	order_date
1001	201	1500.0	2024-01-05

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

order_id	customer_id	amount	order_date
1004	201	3200.0	2024-02-14
1005	204	1100.0	2024-03-01
1011	204	680.0	2024-05-10
1014	201	2700.0	2024-06-15
1015	209	500.0	2024-06-28
1020	209	2200.0	2024-08-14

Q21. Using EXISTS, find all employees who are managers (i.e., at least one other employee has their emp_id as manager_id). Return emp_id, emp_name, and department, ordered by emp_name.

Easy | Topic: EXISTS

Expected Output:

emp_id	emp_name	department
101	Alice Johnson	IT
103	Carol White	HR
105	Eva Green	Finance
108	Henry Wilson	Marketing

Q22. Using NOT EXISTS, find all employees who are NOT managers — meaning no other employee lists them as their manager_id. Return emp_id, emp_name, and department, ordered by emp_id.

Easy | Topic: NOT EXISTS

Expected Output:

emp_id	emp_name	department
102	Bob Smith	IT
104	David Brown	HR
106	Frank Black	Finance
107	Grace Lee	IT
109	Ivy Turner	Marketing
110	Jack Davis	Finance
111	Karen Moore	IT
112	Leo Harris	HR
113	Mia Clark	Marketing
114	Nate Lewis	Finance
115	Olivia Walker	IT
116	Paul Hall	HR

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

emp_id	emp_name	department
117	Quinn Young	Marketing
118	Rachel King	Finance
119	Sam Scott	IT
120	Tina Adams	HR

Q23. Using EXISTS with a correlated subquery on the orders table, find all customers who have placed at least one order. Return customer_id, customer_name, and city, ordered by customer_id.

Easy | Topic: EXISTS

Expected Output:

customer_id	customer_name	city
201	Aarav Mehta	Mumbai
202	Priya Sharma	Delhi
203	Rohan Gupta	Pune
204	Sunita Patel	Ahmedabad
205	Vikram Rao	Bangalore
206	Neha Singh	Chennai
207	Arjun Kumar	Hyderabad
208	Deepa Nair	Kochi
209	Sanjay Joshi	Jaipur
210	Meera Iyer	Coimbatore

Q24. Using NOT EXISTS, find all customers who have never placed any order. Return customer_id, customer_name, and city, ordered by customer_id.

Easy | Topic: NOT EXISTS

Expected Output:

customer_id	customer_name	city
211	Ravi Verma	Lucknow
212	Anita Das	Kolkata
213	Kiran Reddy	Vizag
214	Pooja Shah	Surat
215	Amit Bose	Nagpur
216	Tara Menon	Trivandrum
217	Nikhil Kulkarni	Nashik
218	Swati Chatterjee	Bhopal

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

customer_id	customer_name	city
219	Rahul Pandey	Patna
220	Divya Tiwari	Indore

Q25. Using EXISTS with a correlated subquery on the orders table, find all products that appear in at least one order. Return product_id, product_name, and category, ordered by product_id.

Easy | Topic: EXISTS

Expected Output:

product_id	product_name	category
301	Laptop	Electronics
302	Smartphone	Electronics
303	Headphones	Electronics
304	Monitor	Electronics

Q26. Find all employees (excluding emp_id 105 — Eva Green herself) who work in the same department as emp_id 105. Use a single-row subquery to get Eva Green's department. Return emp_name, department, and salary ordered by salary descending.

Easy | Topic: Single-Row Subquery

Expected Output:

emp_name	department	salary
Rachel King	Finance	82000
Nate Lewis	Finance	78000
Frank Black	Finance	75000
Jack Davis	Finance	70000

Q27. Find the order_id, customer_id, amount, and order_date of the most recently placed order (the order with the latest order_date in the table). Use a single-row subquery returning MAX(order_date).

Easy | Topic: Single-Row Subquery

Expected Output:

order_id	customer_id	amount	order_date
1020	209	2200.0	2024-08-14

Q28. Find the order_id, customer_id, amount, and order_date of the earliest order ever placed (the order with the minimum order_date). Use a single-row subquery.

Easy | Topic: Single-Row Subquery

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

Expected Output:

order_id	customer_id	amount	order_date
1001	201	1500.0	2024-01-05

Q29. Find all employees whose salary is greater than the salary of EVERY employee in the HR department. In other words, their salary must exceed even the highest-paid HR employee. Return emp_name, department, and salary ordered by salary descending. (Use a scalar subquery returning MAX salary of HR.)

Easy | Topic: Single-Row Subquery

Expected Output:

emp_name	department	salary
Rachel King	Finance	82000
Eva Green	Finance	80000
Nate Lewis	Finance	78000
Frank Black	Finance	75000
Alice Johnson	IT	72000
Olivia Walker	IT	71000
Jack Davis	Finance	70000
Grace Lee	IT	68000
Sam Scott	IT	66000
Bob Smith	IT	65000
Karen Moore	IT	63000
Mia Clark	Marketing	62000
Henry Wilson	Marketing	60000
Quinn Young	Marketing	59000

Q30. Among orders with status = 'Delivered', find those whose amount is greater than the average amount of all Delivered orders. Use a single-row scalar subquery to compute the average of Delivered orders. Return order_id, customer_id, amount, and status ordered by amount descending.

Easy | Topic: Single-Row Subquery

Expected Output:

order_id	customer_id	amount	status
1006	205	4500.0	Delivered
1018	210	4100.0	Delivered
1016	206	3800.0	Delivered
1010	207	3300.0	Delivered
1004	201	3200.0	Delivered

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

order_id	customer_id	amount	status
1012	208	2900.0	Delivered

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

Section 3 — Intermediate Questions (Q31–Q45)

📌 **Correlated Subquery · EXISTS/NOT EXISTS · Nested IN · ANY / ALL**

Q31. Using a correlated subquery, find all employees who earn more than the average salary of their own department. For each qualifying employee, also display their department's average salary (rounded to 2 decimal places). Order by department, then salary descending.

Intermediate | Topic: Correlated Subquery

Expected Output:

emp_name	department	salary	dept_avg
Rachel King	Finance	82000	77000.0
Eva Green	Finance	80000	77000.0
Nate Lewis	Finance	78000	77000.0
Carol White	HR	58000	54000.0
Tina Adams	HR	55000	54000.0
Alice Johnson	IT	72000	67500.0
Olivia Walker	IT	71000	67500.0
Grace Lee	IT	68000	67500.0
Mia Clark	Marketing	62000	59250.0
Henry Wilson	Marketing	60000	59250.0

Q32. For every employee, use a correlated subquery to count how many other employees in the same department earn strictly more than them. Display emp_name, department, salary, and this count as 'higher_earners'. Order by department, salary descending.

Intermediate | Topic: Correlated Subquery

Expected Output:

emp_name	department	salary	higher_earners
Rachel King	Finance	82000	0
Eva Green	Finance	80000	1
Nate Lewis	Finance	78000	2
Frank Black	Finance	75000	3
Jack Davis	Finance	70000	4
Carol White	HR	58000	0
Tina Adams	HR	55000	1
David Brown	HR	54000	2
Paul Hall	HR	53000	3

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

emp_name	department	salary	higher_earners
Leo Harris	HR	50000	4
Alice Johnson	IT	72000	0
Olivia Walker	IT	71000	1
Grace Lee	IT	68000	2
Sam Scott	IT	66000	3
Bob Smith	IT	65000	4
Karen Moore	IT	63000	5
Mia Clark	Marketing	62000	0
Henry Wilson	Marketing	60000	1
Quinn Young	Marketing	59000	2
Ivy Turner	Marketing	56000	3

Q33. Using a correlated subquery inside the WHERE clause, find all customers whose total order spend (sum of all their order amounts) is greater than the average total spend per customer across all ordering customers. Display customer_id, customer_name, and total_spend. Order by total_spend descending. (Compute per-customer total and the overall average both via subqueries — no window functions.)

Intermediate | Topic: Correlated Subquery

Expected Output:

customer_id	customer_name	total_spend
201	Aarav Mehta	7400.0
206	Neha Singh	5900.0
205	Vikram Rao	5700.0
207	Arjun Kumar	4900.0

Q34. Using a subquery in the WHERE clause with IN and a GROUP BY / HAVING inside the subquery, find all employees who belong to departments where the average salary of the department is greater than 65,000. Return emp_name, department, and salary, ordered by department and salary descending.

Intermediate | Topic: Multi-Row Subquery (IN)

Expected Output:

emp_name	department	salary
Rachel King	Finance	82000
Eva Green	Finance	80000
Nate Lewis	Finance	78000
Frank Black	Finance	75000
Jack Davis	Finance	70000

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

emp_name	department	salary
Alice Johnson	IT	72000
Olivia Walker	IT	71000
Grace Lee	IT	68000
Sam Scott	IT	66000
Bob Smith	IT	65000
Karen Moore	IT	63000

Q35. For every row in the orders table, use a correlated subquery inside a CASE expression to label each order as 'Above Avg' if its amount is greater than or equal to that customer's own average order amount, and 'Below Avg' otherwise. Show order_id, customer_id, amount, and the label as 'vs_cust_avg'. Order by customer_id, order_id.

Intermediate | Topic: Correlated Subquery

Expected Output:

order_id	customer_id	amount	vs_cust_avg
1001	201	1500.0	Below Avg
1004	201	3200.0	Above Avg
1014	201	2700.0	Above Avg
1002	202	2300.0	Above Avg
1007	202	950.0	Below Avg
1003	203	800.0	Below Avg
1009	203	1750.0	Above Avg
1005	204	1100.0	Above Avg
1011	204	680.0	Below Avg
1006	205	4500.0	Above Avg
1013	205	1200.0	Below Avg
1008	206	2100.0	Below Avg
1016	206	3800.0	Above Avg
1010	207	3300.0	Above Avg
1017	207	1600.0	Below Avg
1012	208	2900.0	Above Avg
1019	208	720.0	Below Avg
1015	209	500.0	Below Avg
1020	209	2200.0	Above Avg
1018	210	4100.0	Above Avg

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

Q36. Using a correlated subquery, find all products whose price is greater than the average price of other products in the same category. Display product_name, category, price, and the category average (rounded to 2 decimals) as 'cat_avg'. Order by category, price descending.

Intermediate | Topic: Correlated Subquery

Expected Output:

product_name	category	price	cat_avg
Keyboard	Accessories	1500	1050.0
USB Hub	Accessories	1200	1050.0
Laptop	Electronics	55000	14900.0
Smartphone	Electronics	22000	14900.0
Monitor	Electronics	18000	14900.0
Smart Watch	Electronics	15000	14900.0
Standing Desk	Furniture	22000	10900.0
Pen Set	Stationery	250	200.0

Q37. Find all employees whose salary is greater than the salary of AT LEAST ONE employee in the Marketing department. Use ANY (or equivalently, > MIN of Marketing salaries). Return emp_name, department, and salary ordered by salary descending. Note: Ivy Turner (56,000) is the lowest-paid Marketing employee — so all employees earning above 56,000 qualify.

Intermediate | Topic: Multi-Row Subquery (ANY)

Expected Output:

emp_name	department	salary
Rachel King	Finance	82000
Eva Green	Finance	80000
Nate Lewis	Finance	78000
Frank Black	Finance	75000
Alice Johnson	IT	72000
Olivia Walker	IT	71000
Jack Davis	Finance	70000
Grace Lee	IT	68000
Sam Scott	IT	66000
Bob Smith	IT	65000
Karen Moore	IT	63000
Mia Clark	Marketing	62000
Henry Wilson	Marketing	60000
Quinn Young	Marketing	59000
Carol White	HR	58000

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

Q38. Find all employees whose salary is less than the salary of EVERY employee in the Finance department. In other words, their salary must be below even the minimum Finance salary (70,000). Use ALL (or < MIN of Finance). Return emp_name, department, and salary ordered by salary descending.

Intermediate Topic: Multi-Row Subquery (ALL)

Expected Output:

emp_name	department	salary
Grace Lee	IT	68000
Sam Scott	IT	66000
Bob Smith	IT	65000
Karen Moore	IT	63000
Mia Clark	Marketing	62000
Henry Wilson	Marketing	60000
Quinn Young	Marketing	59000
Carol White	HR	58000
Ivy Turner	Marketing	56000
Tina Adams	HR	55000
David Brown	HR	54000
Paul Hall	HR	53000
Leo Harris	HR	50000

Q39. Find customers who have placed at least one order AND whose every order has status = 'Delivered' (i.e., they have no Cancelled or Pending orders). Use a combination of EXISTS and NOT EXISTS. Return customer_id and customer_name ordered by customer_id.

Intermediate Topic: EXISTS / NOT EXISTS

Expected Output:

customer_id	customer_name
205	Vikram Rao
206	Neha Singh
207	Arjun Kumar
209	Sanjay Joshi
210	Meera Iyer

Q40. Using nested IN subqueries, find all products that were ordered by at least one customer from the city of Mumbai. First find customer_ids from Mumbai, then find product_ids ordered by those customers, then return the product details. Show product_id, product_name, and category, ordered by product_id.

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

Intermediate	Topic: Multi-Row Subquery (Nested IN)

Expected Output:

product_id	product_name	category
301	Laptop	Electronics
302	Smartphone	Electronics
304	Monitor	Electronics

Q41. Find the employee(s) with the second-highest salary in the entire company. Use a subquery that first finds the maximum salary, then an outer query that finds the maximum salary below that value. Return emp_name, department, and salary.

Intermediate	Topic: Single-Row Subquery (Nested)

Expected Output:

emp_name	department	salary
Eva Green	Finance	80000

Q42. Using NOT EXISTS, find all departments where every single employee earns strictly more than 50,000. (i.e., there is no employee in that department with salary <= 50,000.) Return just the department name, ordered alphabetically. Note: Leo Harris in HR earns exactly 50,000, so HR does not qualify.

Intermediate	Topic: NOT EXISTS (Correlated)

Expected Output:

department
Finance
IT
Marketing

Q43. Using a correlated subquery in the WHERE clause, find all customers who have placed orders for at least 2 different products (i.e., their orders contain at least 2 distinct product_id values). Return customer_id and customer_name, ordered by customer_id.

Intermediate	Topic: Correlated Subquery

Expected Output:

customer_id	customer_name
201	Aarav Mehta
202	Priya Sharma
203	Rohan Gupta
204	Sunita Patel
205	Vikram Rao

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

customer_id	customer_name
207	Arjun Kumar
208	Deepa Nair
209	Sanjay Joshi

Q44. Using a correlated subquery, find all employees whose salary is above the average salary of all employees hired in the same year as them. Show emp_name, department, salary, hire_year, and the year's average (rounded to 2 dp) as 'year_avg'. Order by hire_year, salary descending.

Intermediate | Topic: Correlated Subquery

Expected Output:

emp_name	department	salary	hire_year	year_avg
Rachel King	Finance	82000	2016	81000.0
Olivia Walker	IT	71000	2017	63000.0
Frank Black	Finance	75000	2018	69666.67
Alice Johnson	IT	72000	2018	69666.67
Nate Lewis	Finance	78000	2019	67000.0
Jack Davis	Finance	70000	2019	67000.0
Karen Moore	IT	63000	2020	56666.67
Grace Lee	IT	68000	2021	59000.0
Sam Scott	IT	66000	2022	61000.0

Q45. Using a correlated subquery, find all products whose stock_qty is less than the average stock_qty of all products in the same category. Display product_name, category, stock_qty, and the category average stock rounded to 2 dp as 'cat_avg_stock'. Order by category, stock_qty ascending.

Intermediate | Topic: Correlated Subquery

Expected Output:

product_name	category	stock_qty	cat_avg_stock
USB Hub	Accessories	180	232.5
Keyboard	Accessories	200	232.5
Scanner	Electronics	20	60.5
Printer	Electronics	25	60.5
Laptop	Electronics	30	60.5
Monitor	Electronics	45	60.5
Smart Watch	Electronics	55	60.5
External SSD	Electronics	60	60.5
Standing Desk	Furniture	15	48.33

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

product_name	category	stock_qty	cat_avg_stock
Desk Chair	Furniture	40	48.33
Notebook	Stationery	500	550.0

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

Section 4 — Hard Questions (Q46–Q60)

📌 **Complex Correlated · Nested Subqueries · Multi-level EXISTS · Subquery in HAVING**

Q46. Find all employees whose salary is among the top 3 distinct salary values in the company. For example, if the top 3 distinct salaries are 82000, 80000, and 78000, return all employees earning any of those values. Use a subquery with LIMIT or a nested approach — no window functions.

Hard	Topic: Multi-Row Subquery (IN + LIMIT)

💡 **Approach Hint:**

→ First find all distinct salary values ordered descending, and limit to the top 3.
→ Then use IN (or a comparison) to filter the employees table to those 3 salary values.
→ Think: what does 'SELECT DISTINCT salary ORDER BY salary DESC LIMIT 3' return, and how do you use it?

Expected Output:

emp_name	department	salary
Rachel King	Finance	82000
Eva Green	Finance	80000
Nate Lewis	Finance	78000

Q47. Find all customers (who have placed at least one order) where every single one of their orders has an amount strictly greater than 1,000. Customers with any order at or below 1,000 must be excluded. Use EXISTS and NOT EXISTS. Return customer_id and customer_name, ordered by customer_id.

Hard	Topic: EXISTS / NOT EXISTS

💡 **Approach Hint:**

→ Use EXISTS to confirm the customer has at least one order.
→ Use NOT EXISTS to confirm there is no order for that customer with amount <= 1000.
→ Both conditions must be true simultaneously for the customer to appear.

Expected Output:

customer_id	customer_name
201	Aarav Mehta
205	Vikram Rao
206	Neha Singh
207	Arjun Kumar
210	Meera Iyer

Q48. For each department, find the employee whose salary is closest to that department's average salary (minimum absolute difference). If two employees are equally close, both should appear. Show emp_name, department, salary, dept_avg (rounded to 2 dp), and the difference (diff). Order by department.

Hard	Topic: Correlated Subquery (nested)

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

💡 **Approach Hint:**

→ For each employee, compute the absolute difference between their salary and their department's average (use a correlated subquery for the department average).

→ Then, for each department, find the minimum of those absolute differences (another correlated subquery scanning the same department).

→ Keep only the employees whose individual difference equals the minimum difference for their department.

Expected Output:

emp_name	department	salary	dept_avg	diff
Nate Lewis	Finance	78000	77000.0	1000.0
David Brown	HR	54000	54000.0	0.0
Grace Lee	IT	68000	67500.0	500.0
Quinn Young	Marketing	59000	59250.0	250.0

Q49. Find all customers (other than customer 201) who have ordered every product that customer 201 has ordered. Customer 201 (Aarav Mehta) has ordered product_ids 301, 302, and 304. A candidate customer must have ordered all three. Use a NOT EXISTS / EXCEPT approach. Return customer_id and customer_name ordered by customer_id. (With this dataset, the expected result is an empty set.)

Hard	Topic: NOT EXISTS / EXCEPT

💡 **Approach Hint:**

→ Think of it as: 'There does not exist any product that customer 201 ordered, which the candidate customer has NOT ordered.'

→ Use a NOT EXISTS wrapping an inner query that looks for products in customer 201's orders that are missing from the candidate's orders.

→ The EXCEPT set operator (or a NOT IN with a correlated filter) can express 'products ordered by 201 but not by X'.

Expected Output:

customer_id	customer_name
(No rows — no other customer has ordered all three products 301, 302, and 304)	

Q50. Find all departments whose total salary bill is greater than the total salary bill of at least one other department (i.e., greater than the minimum department total). Show department and dept_total ordered by dept_total descending. Use a subquery in the HAVING clause.

Hard	Topic: Subquery in HAVING

💡 **Approach Hint:**

→ Compute each department's total salary via a correlated subquery or a GROUP BY subquery.

→ In the HAVING clause, compare the department total against a subquery that returns the minimum total across all departments.

→ A department qualifies if its total is strictly greater than that minimum.

Expected Output:

department	dept_total
IT	405000
Finance	385000

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

department	dept_total
HR	270000

Q51. Using only subqueries (no self-join syntax), find all employees who earn strictly more than their own manager's salary. For each such employee, display emp_name, their salary as emp_salary, their manager's name as manager_name, and the manager's salary as manager_salary. Order by emp_salary descending.

Hard	Topic: Correlated Scalar Subquery

💡 **Approach Hint:**

→ For each employee with a non-NULL manager_id, use a scalar subquery to look up their manager's salary.

→ Filter using WHERE emp.salary > (SELECT salary FROM employees WHERE emp_id = outer.manager_id).

→ Use additional scalar subqueries in the SELECT list to retrieve the manager's name.

Expected Output:

emp_name	emp_salary	manager_name	manager_salary
Rachel King	82000	Eva Green	80000
Mia Clark	62000	Henry Wilson	60000

Q52. Find all products whose total number of orders is greater than the average number of orders per product (across only those products that appear in at least one order). Use a correlated subquery to count each product's orders, and a non-correlated subquery to compute the average order count per product. Show product_id, product_name, and order_count.

Hard	Topic: Correlated Subquery

💡 **Approach Hint:**

→ Use (SELECT COUNT(*) FROM orders WHERE product_id = p.product_id) to get each product's count.

→ Compute the average order count as a standalone subquery: SELECT AVG(cnt) FROM (SELECT COUNT(*) AS cnt FROM orders GROUP BY product_id).

→ With only 4 products ordered and 20 total orders, the average is 5. Check which products exceed 5 orders.

Expected Output:

product_id	product_name	order_count
(No rows — all 4 ordered products have exactly 5 orders each, none exceed the average of 5)		

Q53. Find the customer(s) whose single highest order amount is equal to the global maximum order amount in the entire orders table. For each such customer, show customer_id, customer_name, and their best_order (maximum order amount). Use correlated scalar subqueries — no joins.

Hard	Topic: Correlated Scalar Subquery

💡 **Approach Hint:**

→ Use a correlated subquery in the SELECT to get each customer's MAX(amount).

→ In the WHERE clause, compare that correlated max against a standalone subquery returning the overall MAX(amount).

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

→ Only customers whose personal best equals the global best will qualify.

Expected Output:

customer_id	customer_name	best_order
205	Vikram Rao	4500.0

Q54. Find all employees who belong to departments that have a budget greater than the average budget across all departments in the departments table. Use a subquery with IN to identify qualifying departments, then retrieve employees. Show emp_name, department, and salary ordered by department and salary descending.

Hard	Topic: Multi-Row Subquery (IN)

💡 **Approach Hint:**

→ Compute the average department budget with a standalone subquery on the departments table.
→ Find dept_names where budget > that average using a subquery.
→ Use IN to filter the employees table against those dept_names.

Expected Output:

emp_name	department	salary
Rachel King	Finance	82000
Eva Green	Finance	80000
Nate Lewis	Finance	78000
Frank Black	Finance	75000
Jack Davis	Finance	70000
Alice Johnson	IT	72000
Olivia Walker	IT	71000
Grace Lee	IT	68000
Sam Scott	IT	66000
Bob Smith	IT	65000
Karen Moore	IT	63000

Q55. For each customer who has placed at least one order, use a correlated subquery to count how many of their own orders have an amount strictly greater than their personal average order amount. Display customer_id, customer_name, and orders_above_avg. Order by orders_above_avg descending, then customer_id.

Hard	Topic: Correlated Subquery (doubly nested)

💡 **Approach Hint:**

→ For each customer, compute their personal average with a correlated subquery inside another correlated subquery.
→ Count the orders where amount > that personal average using COUNT(*) in the outer correlated subquery.
→ The WHERE clause on the outer query should use EXISTS to restrict to customers who have at least one order.

Expected Output:

customer_id	customer_name	orders_above_avg
201	Aarav Mehta	2

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

customer_id	customer_name	orders_above_avg
202	Priya Sharma	1
203	Rohan Gupta	1
204	Sunita Patel	1
205	Vikram Rao	1
206	Neha Singh	1
207	Arjun Kumar	1
208	Deepa Nair	1
209	Sanjay Joshi	1
210	Meera Iyer	0

Q56. Find all products that were ordered exclusively by customers from a single city (i.e., every customer who ever ordered that product is from the same city). Only consider products that appear in the orders table. Show product_id, product_name, and that single city as 'only_city'. With the current data, check whether any product meets this criterion.

Hard	Topic: Correlated Subquery with Aggregation

💡 **Approach Hint:**

→ For each product in orders, use a subquery to count the number of DISTINCT cities of customers who ordered it.
→ Join orders with customers inside the subquery to get city information.
→ Keep products where that distinct city count equals 1.

Expected Output:

product_id	product_name	only_city
(No rows — all 4 ordered products were bought by customers from multiple cities)		

Q57. Find employees who are the sole highest earner in their department — meaning they earn the maximum salary in their department AND no other employee in that department earns the same amount. Show emp_name, department, and salary ordered by salary descending.

Hard	Topic: Correlated Subquery (double condition)

💡 **Approach Hint:**

→ Use a subquery to find the MAX salary in each employee's department.
→ Use a second correlated subquery to count how many employees in that department share that maximum salary.
→ Keep only employees where the count equals 1.

Expected Output:

emp_name	department	salary
Rachel King	Finance	82000
Alice Johnson	IT	72000

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

emp_name	department	salary
Mia Clark	Marketing	62000
Carol White	HR	58000

Q58. Find customers whose total spend is more than double the average total spend per customer. Use a subquery to compute each customer's total spend and another to compute the average of those totals. Show customer_id, customer_name, and total_spend. With the current data, verify whether any customer meets this criterion.

Hard Topic: Correlated Subquery (derived table avg)

💡 **Approach Hint:**

→ Compute each customer's total spend with a correlated subquery: SUM(amount) WHERE customer_id = c.customer_id.

→ Compute the average of all customer totals with a non-correlated subquery over a derived table.

→ The threshold is 2 × average. Check if any customer's total exceeds that threshold.

Expected Output:

customer_id	customer_name	total_spend
(No rows — no customer's total spend exceeds 2× the average customer spend of ≈3895)		

Q59. Find the department that has the highest count of employees earning above the company-wide average salary. Use a correlated subquery to count above-average earners per department, and return only the top department. Show department and above_avg_count.

Hard Topic: Correlated Subquery in SELECT + HAVING

💡 **Approach Hint:**

→ Compute company-wide average salary with a standalone scalar subquery.

→ For each department, use a correlated subquery to count employees in that department whose salary > company average.

→ Group by department, and use LIMIT 1 with ORDER BY to get the department with the highest count.

Expected Output:

department	above_avg_count
IT	5

Q60. Find all employees who are the highest earner in their own department (salary = dept MAX) but whose overall salary rank in the company is beyond position 3 — meaning at least 3 other distinct salary values in the company are higher than theirs. Show emp_name, department, and salary ordered by salary descending.

Hard Topic: Correlated Subquery (multi-condition)

💡 **Approach Hint:**

→ Use a correlated subquery to check that the employee's salary equals their department's MAX salary.

→ Use a second subquery to count how many distinct salary values in the employees table are strictly greater than this employee's salary.

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

→ Keep only employees where that count is >= 3, meaning they are not in the overall top 3 salary positions.

Expected Output:

emp_name	department	salary
Alice Johnson	IT	72000
Mia Clark	Marketing	62000
Carol White	HR	58000

Registered Office:
S6,7,8, VS TOWER, II Floor,
Mahaveer Nagar, Main Tonk Road,
Jaipur, Rajasthan 302018

Notes

- All solutions are in the companion file: SQL_60Q_Solutions.docx
- SQL dialect: MySQL 8+ / PostgreSQL. For SQLite, EXCEPT is supported; ALL/ANY uses scalar subquery equivalents.
- Easy questions introduce single-row and multi-row subquery patterns one concept at a time.
- Intermediate questions combine correlated subqueries, EXISTS/NOT EXISTS, and nested IN.
- Hard questions require multiple nested or doubly-correlated subqueries — read the hint steps carefully.
- Expected outputs were verified by running each SQL against a real SQLite database with the exact data shown above.